

LSHiForest: A Generic Framework for Fast Tree Isolation based Ensemble Anomaly Analysis

Xuyun Zhang
The University of Auckland
xuyun.zhang@auckland.ac.nz

Wanchun Dou
Nanjing University
douwc@nju.edu.cn

Qiang He
Swinburne University of Technology
qhe@swin.edu.au

Rui Zhou
Victoria University
rui.zhou@vu.edu.au

Christopher Leckie
The University of Melbourne
caleckie@unimelb.edu.au

Ramamohanarao Kotagiri
The University of Melbourne
kotagiri@unimelb.edu.au

Zoran Salcic
The University of Auckland
z.salcic@auckland.ac.nz

Abstract—Anomaly or outlier detection is a major challenge in big data analytics because anomaly patterns provide valuable insights for decision-making in a wide range of applications. Recently proposed anomaly detection methods based on the tree isolation mechanism are very fast due to their logarithmic time complexity, making them capable of handling big data sets efficiently. However, the underlying similarity or distance measures in these methods have not been well understood. Contrary to the claims that these methods never rely on any distance measure, we find that they have close relationships with certain distance measures. This implies that the current use of this fast isolation mechanism is only limited to these distance measures and fails to generalise to other commonly-used measures. In this paper, we propose a generic framework named LSHiForest for fast tree isolation based ensemble anomaly analysis with the use of a Locality-Sensitive Hashing (LSH) forest. Being generic, the proposed framework can be instantiated with a diverse range of LSH families, and the fast isolation mechanism can be extended to any distance measures, data types and data spaces where an LSH family is defined. In particular, the instances of our framework with kernelised LSH families or learning based hashing schemes can detect complicated anomalies like local or surrounded anomalies. We also formally show that the existing tree isolation based detection methods are special cases of our framework with the corresponding distance measures. Extensive experiments on both synthetic and real-world benchmark data sets show that the framework can achieve both high time efficiency and anomaly detection quality.

I. INTRODUCTION

Anomaly detection or outlier detection aims to automatically identify or predict anomalous but insightful patterns in data sets. It is an important and powerful data mining technique which is widely adopted in a diverse range of applications such as fraud detection, system health monitoring and event detection in sensor networks. Throughout this paper we use the terms ‘anomaly’ and ‘outlier’ interchangeably as the definition of an anomaly or outlier remains rather vague. A commonly adopted one is “an observation which deviates so much from other observations as to arouse suspicions that it was generated by a different mechanism” [1], e.g., abnormally high temperature readings in environmental sensing data may indicate an occurrence of bushfire. A wide range of methods have been proposed based on different models of anomalies in various domains [2], [3]. Some seminal examples are clustering based outlier detection [4], distance (or density) based outlier

detection [5], relative density based outlier detection [6], angle based outlier detection [7] and isolation based outlier detection [8]. One can refer to [2], [3] for more details.

In the current big data era, data sets with large-volume, high-dimensional, heterogeneous, geographically distributed and fast-evolving characteristics pose a considerable challenge on traditional data mining and analytics techniques including anomaly detection [9]. The problem of anomaly detection on big data sets becomes even harder as we need to find ‘needles’ in a much larger and more complex ‘haystack’. Specifically, the large-volume and high-dimensional features usually disable traditional algorithms having quadratic (or higher) computational complexity as they often fail to produce meaningful results within a “tolerable elapsed time”. Hence, the detection algorithms for big data are often required to be highly scalable. Besides, most anomaly detection applications such as network intrusion detection and human health monitoring, demand low-latency or even real-time detection or prediction to trigger a fast response in the case of anomalous events. As such, scalability and efficiency are of paramount importance for the success of anomaly detection in big data applications.

Given these challenges, subsampling based ensemble methods are believed to be a promising way to enhance the efficiency and performance of existing detection algorithms for big data. Ensemble methods have been widely adopted as a meta-algorithm for many data mining tasks, especially for classification and clustering, aiming to reduce the dependence of models on specific datasets or data locality, and enhance the robustness of a data mining process [10]. As to anomaly detection, however, it has been recently pointed out that ensemble outlier analysis is an emerging research area and still in its infancy due to the intrinsic nature of the outlier analysis problem [11], [12]. The challenge of anomaly detection is mainly twofold, i.e., small sample space (a given data set usually contains only a small fraction of outliers) and unsupervised nature (lack of ground truth) [11]. Thus, few efforts have been made to apply ensemble methods in improving the detection performance in a principled way. The work in [11], [13] has initially formulated the key algorithmic ingredients of ensemble outlier analysis, and developed the theoretical foundation from the perspective of the well-established bias-variance trade-off theory [14].

Compared to classification and clustering, the variety of

ensemble based anomaly detection algorithms is still limited and the corresponding work in the literature is relatively complementary and sporadic. The work on feature bagging in [15] is regarded as the first attempt to use ensemble for anomaly detection in a formal and explicit way, while some earlier work using ensembles implicitly also exists as pointed out in [11]. Orthogonal to feature bagging or subspace ensemble methods, a subsampling based ensemble is leveraged in [16] to improve the performance and efficiency of the seminal Local Outlier Factor (LOF) analysis [6]. However, the computational complexity of predicting the anomaly scores of each data instance is at least linear in these algorithms with respect to the sample size, making the whole detection or prediction process quite time-consuming in the case of large-scale data. The salient work in [8] proposes iForest, an anomaly detection method that leverages a random forest to isolate data instances and derives anomaly scores based on path lengths. This method is very fast because the average computational complexity at the evaluation stage is logarithmic with respect to the sample size. To avoid confusion, we call this isolation mechanism as tree isolation because it uses a tree structure to isolate data instances. Note that another isolation mechanism based on building hyperspheres has been proposed for anomaly detection [17], but it has high computational complexity. As pointed out in [17], [18], iForest suffers from an incapability in detecting complicated anomalies such as axis-parallel, local or surrounded anomalies. Its variant SCiForest [18] uses multiple attributes at each step of data partitioning and an optimisation process to mitigate the drawbacks of iForest. However, the distance or similarity measures underlying these two methods have not been well understood since the authors inappropriately claimed that both methods never rely on any distance measure. As analysed in Section IV-C, iForest and SCiForest have close relationships with the ℓ_1 or Manhattan distance and an angular distance measure, respectively. As a result, the current use of the fast tree isolation mechanism is limited to these two distance measures, and fails to generalise to other commonly-used distance measures, data types or data spaces.

In this paper, we investigate the tree isolation mechanism for anomaly detection in a principled way and propose a generic detection framework named LSHiForest, with the use of the Locality-Sensitive Hashing (LSH) forest data structure [19]. The basic idea is that we make use of the salient LSH mechanism to achieve fast data instance isolation with the same logarithmic computational complexity as the existing methods [8], [18]. But more importantly, our approach can appropriately incorporate the nearest neighbourhood information of data instances into the isolation process, while such information used in iForest is rather coarse or limited. Another crucial impact is that our approach makes the isolation mechanism more versatile and flexible as it can be extended into a much wider scope given the recent fruitful achievements in LSH research [20]. Specifically, we formulate the algorithmic details of the generic framework for ensemble anomaly analysis. Then, iForest and SCiForest are revisited and formally shown to be special cases of our framework. Typical instances of the framework are developed for three different LSH families, i.e., the ℓ_p ($p = 1, 2$) LSH (for the Manhattan and Euclidean distances) [21], the angle-based LSH [22], and the kernelised LSH (for kernelised space) [23] to demonstrate the versatility of our approach. Note that the generic framework can be

applied anywhere as an LSH family is defined, e.g., Min-hash LSH for set-valued or categorical data [24]. One can instantiate the framework with corresponding LSH families to detect anomalies in the desired distance space. Extensive experiments are conducted on both synthetic and real-world benchmark data sets, and the results show that our approach can significantly improve the detection performance and time efficiency of ensemble anomaly analysis over existing methods.

The main contributions of our approach are fourfold. First, a generic framework with LSH forest isolation is proposed for fast and versatile ensemble anomaly analysis. Then, we formally show that the existing tree isolation based methods are special cases of our framework. The framework is developed with several commonly-used LSH families to demonstrate its versatility. Finally, we conduct extensive experiments on both synthetic and real-world benchmark data sets to validate the effectiveness and efficiency of our approach.

The remainder of this paper is organised as follows. The next section reviews the related research work. In Section III, we describe the problem setting and discuss our research motivation. Our generic framework and its instantiations are formulated in Section IV, followed by extensive empirical evaluation on benchmark data sets in Section V. We conclude this paper and discuss future work in Section VI.

II. RELATED WORK

As pointed out in [11], a range of previous work attains ensemble anomaly analysis implicitly by combining anomaly scores from either multiple independent component models or dependent component models (single or multiple) running sequentially on the same data. These are orthogonal to the ensemble method our approach follows herein, where different samples of the data are explored to perform anomaly analysis. Interested readers could refer to [11] for more details. In this section, we briefly survey the research work closely relevant to our approach, i.e., subspace or subsampling based ensemble anomaly analysis, subsampling based anomaly detection and locality-sensitive hashing (LSH) related outlier detection.

Lazarevic and Kumar [15] proposed the *feature bagging* technique for anomaly detection where the subspace ensemble method is explicitly and formally elaborated. This approach generates a number of subspaces of the data and combines the anomaly scores of data instances derived from these projected subspaces as the final scores. Zimek et al. [16] proposed a subsampling based approach for ensemble anomaly detection, which derives the anomaly scores of a data instance from subsamples of original data. It is reported in [16] that the subsampling based ensemble method can improve the quality of outlier detection and reduce the computational cost considerably compared with the *feature bagging* technique [15] or just the single component algorithm (LOF [6]) on the complete data. Furthermore, Aggarwal and Sathe [13] exploited the bias-variance trade-off theory [14] to formally underpin ensemble anomaly detection methods, and accordingly proposed to use fixed sublinear dimensionality for random subspaces and small constant subsample sizes, as opposed to the settings in [15] and [16], respectively. However, the evaluation stage in these methods can be still very time-consuming when the data scale is large because the time complexity of testing a single data instance is at least linear with respect to the sample size.

Liu et al. [8] proposed an isolation forest based ensemble anomaly detection approach (iForest), where a forest of isolation trees are constructed from training samples. The anomaly scores of a data instance are derived in terms of its path lengths in the trees. The basic idea of iForest is that anomalies often have shorter path lengths and can thus be identified. This method has a salient feature that the time complexity of evaluating a data instance is logarithmic with respect to the sample size, enabling it to handle large-scale data efficiently. However, it is pointed out in [17], [18] that iForest suffers from an inability to detect complicated anomalies such as axis-parallel, local and surrounded anomalies. The work in [18] extended iForest for clustered anomalies and proposed a variant SCiForest, which selects the best one out of a set of randomly generated subspace hyperplanes and the best splitting point corresponding to this hyperplane. The computational cost of the training stage increases accordingly because of the search of the best hyperplane and the best splitting point. Our work also follows the tree isolation mechanism, but our generic framework uses LSH forest to achieve isolation and can be instantiated with a wide range of LSH families. Also, note that iForest and SCiForest are special cases of our framework. Bandaragoda et al. [17] uses nearest neighbours to achieve hypersphere isolation for anomaly detection and proposed a method iNNE (isolation using Nearest Neighbour Ensemble) based on the relative density measure [6], aiming to detect local or surrounded anomalies. However, it is essentially similar to ensemble LOF [16] with a special setting and has the same high computational complexity. Furthermore, their experimental results fail to show a performance improvement over the ensemble LOF method. It is worth noting that the ensemble performance of average k -NN distance is better than that of LOF in most cases as empirically reported in [13], while the latter often performs better in the non-ensemble context. Aggarwal and Sathe [13] further explained this surprising phenomenon well by the bias-variance trade-off theory. As such, it still makes (or even more) sense to develop distance-based detection methods for ensemble anomaly analysis.

Sampling is a simple but effective technique for handling large-scale data, widely adopted in the database and data mining communities. Some sampling based anomaly detection methods without ensembles also exist. Kollois et al. [25] proposed a density biased sampling approach for distance-based outlier detection by oversampling sparse regions. But sampling just works as a pruning process without a rigorous treatment of the effect on result quality [26]. Wu and Jermaine [26] proposed a simple iterative sampling approach, where a sample is drawn for a data instance and the distance of the instance to its k^{th} nearest neighbour (k -NN) in the sample is reported as its anomaly score. Similarly, Sugiyama and Borgwardt [27] proposed a detection approach using only one sample to estimate anomaly scores of all data instances in a data set. Our work is orthogonal to these methods and the sampling strategies can be complementarily applied to our ensemble method. Note that the performance of a base detector running on a complete data set is usually not as good as an ensemble of the base detector in terms of the bias-variance trade-off theory [13].

LSH has been recently exploited in anomaly detection tasks due to its salient features. Wang et al. [28] proposed a ranking based outlier detection approach using LSH to identify

low-density regions for refining the data instances in these regions. Pillutla et al. [29] used LSH to prune strong inliers based on the redundancy of a point in hash tables such that fewer points are used to identify true outliers. The space cost of this approach is a problem for large data sets, as it requires constructing a number of hash tables for redundancy computation. In both methods, LSH only plays a secondary role as a pruning technique and its parameter tuning is critical to detection performance. In contrast, we leverage the LSH forest structure as the core part of our framework, and no parameter tuning is required for an LSH forest [19].

III. PROBLEM STATEMENT

Our generic framework is based on the isolation notion proposed in [8], [18], i.e., separating a data instance from the rest in a sample. Hence, the assumption that anomalies are more susceptible to isolation applies herein as well. As the exact notion of an anomaly often varies considerably in different application domains, most existing detection methods only apply to a limited range of data types and anomaly types. To facilitate the discussion of our core idea, we use commonly-used numerical data and point anomalies as exemplary types in our work. The point anomaly concept implies that no relationship is assumed among data instances and an individual instance can be regarded as an anomaly. This type is widely used in the literature for anomaly analysis, and contextual or collective anomaly analysis where instances are correlated can often be reduced to a point anomaly problem [2]. We are concerned with unsupervised, non-parametric methods herein, which do not require labelled training data and are widely applicable. Accordingly, the output will be a ranked list of anomaly scores that allows analysts to determine the most relevant anomalies by a domain-specific threshold.

Our method follows the subsampling based ensemble framework formulated in [11], [13] which is formally underpinned by the bias-variance trade-off theory [14]. Note that all existing subsampling based ensemble methods [8], [16]–[18] also follow this framework, while varying in base detection algorithms and parameter settings. The ensemble anomaly analysis framework consists of four major stages, i.e., drawing a set of subsamples, training a model for each sample by a base algorithm, evaluating each data instance against the trained models, and combining the anomaly scores of an instance from all models as the final. As the performance of existing methods is sensitive to the selection of sample size, we adopt the variable subsampling technique proposed in [13] for our framework to produce subsamples of different sizes for ensemble components. Note that the sample sizes are bounded within a fixed interval irrespective of the entire data size, [50, 1000] used in [13]. The rationale is that it achieves variance reduction not only over the selection of training data, but also over different randomised choices of base detector parameters, e.g., k in k -NN based anomaly detection. Another advantage is that the users' burden of parameter selection is significantly relieved, while it is usually a challenging task to find the optimal parameters that are usually data-dependent as shown in [8], [13], [16], [17]. Furthermore, we use the widely-adopted averaging method for anomaly score combination. Note that other subsampling, subspace or score combination techniques discussed in [13] can also work well with our generic framework, as our work aims to propose a novel

category of base detection methods for ensemble. We describe the symbols and notations used throughout the paper in Table I.

TABLE I: Symbols and notation.

x	a data instance
X	a data set
n	number of data instances in a data set, $n = X $
m	number of dimensions
t	number of ensemble components
S	a subsample of X , $S \in X$
ψ	sample size
k	the number of nearest neighbours
η	granularity adjustment factor
v	branching factor
$f(\cdot)$	hash function
$\mathcal{F}$	hash function family
$h(\cdot)$	path length
H	tree height

The average-case time complexity of existing subsampling based ensemble anomaly analysis methods are presented in Table II to illustrate the computational advantages of forest based isolation. We use the average-case complexity herein in order to estimate the performance more tightly as the worst-case behaviours rarely occur in practice. Note that the worst-case complexity of the first three methods is the same as the average case, while that of the last two is linear with respect to the sample size ψ . For SciForest, τ is the number of hyperplanes tried in each split and q , $1 \leq q \leq m$ is the number of attributes used to construct a subspace hyperplane [18]. An important observation is that the prediction stage will dominate the execution time when a data set is very large, because n is involved in this stage. Due to the computational advantages of subsampling based ensembles, all methods have linear time complexity with respect to n . Most importantly, the tree isolation based methods take only logarithmic complexity with respect to ψ , while others take linear complexity. This difference makes the former much faster, especially when n is very large. For instance, the speed-up can be 2 orders of magnitude when $\psi = 1000$. Therefore, the tree isolation mechanism offers great computational benefits.

Given the appealing computational benefits of the tree isolation based anomaly analysis and the aforementioned limitations of the existing methods, an interesting question arises: Can we further extend the fast isolation mechanism to other commonly-used distance or similarity measures (e.g., the Euclidean distance), data spaces (e.g., kernelised space) or data types (e.g., categorical data), with the expectation of

increasing the versatility of the isolation based anomaly analysis and improving the detection performance while retaining the low logarithmic computational complexity? This question motivates our research herein, and we show that the answer is *yes*. Concretely, we propose a generic framework for fast ensemble anomaly analysis based on LSH forest isolation, which significantly extends the fast isolation mechanism into a much wider scope via LSH families, while the existing tree isolation based methods are only applicable to the Manhattan (ℓ_1) or angular distances on numerical data.

IV. ANOMALY DETECTION USING LSH FOREST

A. LSH Forest Preliminary

The core idea of Locality Sensitive Hashing (LSH) is to hash ‘nearby’ data instances into the same bucket with higher probability than instances that are far apart to construct indexes for efficient similarity search [30]. Given a distance metric $d(\cdot, \cdot)$, let $d_1 < d_2$ be two distance values, and $p_1, p_2, 0 \leq p_1, p_2 \leq 1$ be two probability values. A family of functions $\mathcal{F}$ is said to be (d_1, d_2, p_1, p_2) -sensitive if for any function $f \in \mathcal{F}$, $x, y \in X$, the following conditions hold:

- 1). $d(x, y) \leq d_1 \Rightarrow \Pr[f(x) = f(y)] \geq p_1$;
- 2). $d(x, y) \geq d_2 \Rightarrow \Pr[f(x) = f(y)] \leq p_2$.

The event $f(x) = f(y)$ means x and y are hashed into the same bucket in a hash table. It is often required that $p_1 > p_2$ to make an LSH family useful. One salient feature of LSH is that we can drive p_1 and p_2 arbitrarily far apart while keeping d_1 and d_2 fixed, implying that the quality of similarity search can be improved as much as we wish. This can be fulfilled by amplifying an LSH family with the banding technique. Specifically, multiple (α) hash functions are randomly chosen from $\mathcal{F}$ to generate a combined hash key by concatenating the α hash values together, and these hash keys determine if two data instances should fall into the same bucket or not. This improves the precision of similarity search by decreasing false positives while affecting the recall of similarity search negatively. Thus, multiple (β) hash tables are constructed to reduce false negatives. Then, two instances are deemed as similar if any pair of combined keys out of the β hash tables are equal. Let s be the similarity degree between two data instances. The probability they can be regarded as similar is $1 - (1 - s^\alpha)^\beta$, an S-shaped curve with respect to s . Another salient feature of LSH is its linear time complexity for similarity computation, while traditional pair-wise computation requires quadratic complexity. Parallelisation can also be applied to LSH computation. As such, LSH has been extensively studied and widely adopted in diverse applications such as fast approximate nearest neighbour search in the database and data mining communities [20].

As the similarity search quality is sensitive to α , a non-trivial task of parameter tuning is often required to obtain good performance [19]. Accordingly, a data structure named LSH forest is proposed in [19] to mitigate the parameter tuning burden by using variable-length combined keys. The core idea is that the combined key of each data instance is made sufficiently long to ensure each instance has a distinct key. Concretely, a (logical) prefix tree named an LSH tree is constructed on the set of all combined keys, where each leaf

TABLE II: Average-case time complexity of existing methods.

Method	Training	Predicting
EnLOF [13], [16]	N/A	$\Theta(n t k^2 \psi m)$
EnKNN [13]	N/A	$\Theta(n t k \psi m)$
iNNE [17]	$\Theta(t \psi^2 m)$	$\Theta(n t \psi m)$
iForest [8]	$\Theta(t \psi \log_2 \psi)$	$\Theta(n t \log_2 \psi)$
SciForest [18]	$\Theta(t \tau \psi^2 \log_2 \psi q)$	$\Theta(n t \log_2 \psi q)$

node corresponds to a data instance. A hash function from an LSH family is used at an internal node to produce different path tags to guide data partitioning. The path tags from the root to a leaf node compose the combined key of the corresponding data instance. A forest of LSH trees are often constructed for robust approximate nearest neighbour search.

B. Generic Framework: LSHiForest

From the perspective of isolation, an LSH tree can be also deemed as an isolation tree in essence because each data instance is isolated from the rest. Therefore, an LSH forest can be leveraged seamlessly for isolation based ensemble anomaly analysis in the same way as the random binary forest used iForest and SCiForest. More importantly, the fast isolation mechanism can be extended to a much wider scope as an LSH forest can work with any LSH family. This makes our framework very versatile as we can incorporate into the isolation process any similarity measure in any data space where an LSH family is defined. Like in iForest, our generic framework also consists of two stages: the training stage constructs LSH forest for subsamples from a given data set, and the evaluation stage computes anomaly scores of data instances from the built LSH forest.

Given an LSH family, LSH trees are constructed in the training stage by recursively partitioning a subsample $S \in X$ until all data instances are isolated. Algorithm 1 describes how to build a forest of LSH trees. To build a tree, we draw a subsample S_i for the input data in Line 2 by variable subsampling as described in [13]. Specifically, the sampling rate follows the uniform distribution $\mathcal{U}(\min\{1, \frac{50}{n}\}, \max\{1, \frac{1000}{n}\})$, i.e., $50 \leq \psi \leq 1000$, or $\psi = n$ if $n < 50$. However, tree heights generated based on this subsampling technique fail to follow the uniform distribution and prefer larger height values. Therefore, we propose to use a new subsampling technique that engenders uniformly distributed tree heights. Our sampling rate is $\min\{1, \frac{2^s}{n}\}$, where s follows the uniform distribution $\mathcal{U}(6, 10)$. Note that ψ will vary in $[64, 1024]$ if $n \geq 64$, which is quite similar to $[50, 1000]$ used in [13]. Our sampling technique can generate more diversity for an ensemble because the tree height distribution is more balanced. It can be proved that the variance of the tree height in our technique is 1.3333 while that in the previous one is 0.94375 if we roughly assume the tree height is approximately $\log_2(\psi)$ and $64 \leq \psi \leq 1024$. Another benefit is that we can reduce the computational cost as it can also be proved that the sample size expectation $E(\psi)$ in our technique is around 346, while that in the previous one is 544.

Algorithm 1 Constructing LSH Forest

Input: Data set X ; LSH family $\mathcal{F}$; number of LSH trees t .

Output: A forest of LSH trees $\{T_i | 1 \leq i \leq t\}$.

- 1: **for** $i, 1 \leq i \leq t$ **do**
 - 2: $S_i \leftarrow \text{variable_subsampling}(X)$;
 - 3: Compute a height limit: $H_i(\psi_i)$;
 - 4: $T_i \leftarrow \text{LSHiTree}(S_i, \mathcal{F}, H_i, 0)$;
 - 5: **return** $\{T_i\}$.
-

In Line 3, we compute a height limit $H(\psi)$ to prevent an LSH tree from growing arbitrarily large if two data instances are identical or very close. A long (unbounded) single-branch path can be generated because an LSH hash function produces

the same key for two identical instances and excessive hash functions are often required to distinguish two very close instances. Note that iForest also suffers from the same problem for identical instances. From the perspective of computational cost, it is necessary to bound the height of an LSH tree. The height limitation poses little influence on the quality of anomaly detection, because the instances with greater depths are usually strong inliers rather than outliers. Besides, single-branch paths are compressed such that an anomalous instance with such a long path can also be identified by the detection algorithm. But note that the height should be long enough to make instances sufficiently isolated, requiring the upper bound to be as tight as possible. As an LSH tree can be deemed as a digital trie [19], we can use the average height of a digital trie to estimate the height limit. Let us consider random digital tries with n data instances, over a digit alphabet $\{K_1, \dots, K_v\}$, $v \geq 2$. To compose a sequence of digits for instance indexing, K_i is chosen for an element of the sequence with probability p_i , $1 \leq i \leq v$. In terms of [31], the average height of the digital tries is around $\frac{2}{g} \ln(n) + \frac{\gamma - \ln(2)}{g} + 1$, where $\gamma \approx 0.5772$ is the Euler constant and $g = -\ln(\sum_{i=1}^v p_i^2)$. We assume herein that all instances in X are independent, and the hash keys generated by an LSH family are also independent and of equal probability, i.e., $p_i = \frac{1}{v}$. Then, an appropriate upper bound for $H(\psi)$ can be

$$E(H) \approx \frac{2 \ln(\psi)}{\ln(v)} + \frac{\gamma - \ln(2)}{\ln(v)} + 1 \leq 2 \log_2(\psi) + 0.8327. \quad (1)$$

The tree construction subroutine in Line 3 of Algorithm 1 is a recursive process, detailed in Algorithm 2. At each step of partitioning, a hash function f_I from the LSH family is used to produce hash values for all data instances, corresponding to the subroutine `lsh_split`(S, f_I) in Line 6 and 9. The input data are partitioned into non-overlapping subsets associated with hash keys, i.e., $\cup_{i=1}^v S_i = S$ and $f_I(x) = K_i$ if and only if $x \in S_i$. To produce a compact tree without single-branch paths, we repeat splitting the input data until generating multiple hash keys or reaching the tree height limit. The compact tree can be further regarded as a PATRICIA trie [19]. As the hash function index I increases accordingly, it records the path length information of a data instance in the uncompressed digital trie, which is thereby retained for the anomaly prediction stage.

After constructing a forest of LSH trees in the training stage, we can evaluate the anomaly scores of instances in a data set, as described in Algorithm 3. For a data instance $x \in X$, we evaluate its path length $h_i(x)$ against the LSH tree T_i in Line 4 via the subroutine `path_length`, which is elaborated in Algorithm 4. To make path lengths from different trees comparable, we normalise $h_i(x)$ based on a reference path length denoted as $\mu(\psi)$ in Line 5. Unlike iForest which uses a single reference value for all trees, different reference values are required for different trees in our approach because the trees are of different sizes. The reference path length used in iForest is the average path lengths of unsuccessful searches in a binary search tree. However, this fails to apply in our case because the branching factor v is often greater than 2. The estimation of average $h(x)$ for external node terminations is the same as that of the successful searches in PATRICIA tries. According to the analysis in [32], the

Algorithm 2 Constructing LSH Tree: LSHiTree($S, \mathcal{F}, H, I$)**Input:** Input data S ; LSH family $\mathcal{F}$; height limit H ; index I .**Output:** An LSH trees T .

```

1: if  $|S| = 0$  then
2:   return NULL;
3: else if  $|S| = 1$  OR  $I > H$  then
4:   return  $node\{Size \leftarrow |S|, Hash\_Index \leftarrow I, Children \leftarrow \emptyset\}$ ;
5: else
6:    $\{K_1 : S_1, \dots, K_v : S_v\} \leftarrow lsh\_split(S, f_I), f_I \in \mathcal{F}$ 
7:   while  $v = 1$  AND  $I \leq H$  do
8:      $I \leftarrow I + 1$ ;
9:      $\{K_1 : S_1, \dots, K_v : S_v\} \leftarrow lsh\_split(S, f_I)$ ;
10:  if  $I > H$  then
11:    return  $node\{Size \leftarrow |S|, Hash\_Index \leftarrow I\}$ ;
12:  Initialise child node indexing:  $\mathbb{C} \leftarrow \emptyset$ ;
13:  for  $i, 1 \leq i \leq v$  do
14:     $C_i \leftarrow LSHiTree(S_i, \mathcal{F}, H, I + 1)$ 
15:     $\mathbb{C} \leftarrow \mathbb{C} \cup \{K_i : C_i\}$ 
16:  return  $node\{Size \leftarrow |S|, Hash\_Index \leftarrow I, Children \leftarrow \mathbb{C}\}$ .

```

expectation of successful search in PATRICIA tries indexing ψ data instances is approximately $\frac{1}{g_1}(\ln(\psi) + \gamma - \bar{g}_1 + \frac{g_2}{2g_1})$, where $g_1 = -\sum_{i=1}^v p_i \ln(p_i)$, $g_2 = \sum_{i=1}^v p_i \ln^2(p_i)$ and $\bar{g}_1 = -\sum_{i=1}^v p_i \ln(1 - p_i)$. Under the same assumption as the height estimation, i.e., $p_i = \frac{1}{v}$, we approximate $\mu(\psi)$ by

$$\mu(\psi) = \begin{cases} \frac{\ln(\psi) + \ln(v-1) + \gamma}{\ln(v)} - \frac{1}{2}, & \psi > v \\ 1, & 1 < \psi \leq v \\ 0, & o.w. \end{cases} \quad (2)$$

Since the information about v is unknown, we need to estimate it from trained LSH trees. Specifically, we can estimate v for a tree by the average branching factor.

Algorithm 3 Predicating Anomaly Scores**Input:** Test data X ; LSH family $\mathcal{F}$; LSH forest $\{T_i \mid 1 \leq i \leq t\}$; depth adjustment factor η , granularity level: L .**Output:** Anomaly scores $\{AS_x \mid x \in X\}$.

```

1: for  $x \in X$  do
2:    $AS_x \leftarrow 0$ ;
3:   for  $i, 1 \leq i \leq t$  do
4:      $h_i(x) \leftarrow path\_length(x, \mathcal{F}, T_i.root, \eta, L, 0)$ ;
5:   Normalisation:  $AS_x \leftarrow AS_x + 2^{-\frac{h_i(x)}{\mu(\psi_i)}}$ ;
6:    $AS_x \leftarrow AS_x / t$ 
7: return  $\{AS_x\}$ .

```

Like in iForest, we further use the exponential function 2^{-x} , $x \geq 0$ to non-linearly scale the normalised path lengths into the interval $(0, 1]$ in Line 5 of Algorithm 3. But note that the non-linear scaling in iForest fails to affect the order of final anomaly scores because it occurs after the combination of normalised path lengths from ensemble components. In contrast, we conduct the non-linear scaling before the combination, making it contribute to improving diversity for an ensemble. In Line 6, we take the arithmetic mean of non-linearly scaled values as the final anomaly score, i.e., $AS_x = \frac{1}{t} \sum_{i=1}^t 2^{-\frac{h_i(x)}{\mu(\psi_i)}}$. In fact, a final anomaly score from iForest can be regarded as the geometric mean of non-linearly scaled values, i.e.,

$2^{-\frac{\frac{1}{t} \sum_{i=1}^t h_i(x)}{c(\psi)}} = \sqrt[t]{\prod_{i=1}^t 2^{-\frac{h_i(x)}{c(\psi)}}}$, where $c(\psi)$ is the reference path length similar to $\mu(\psi)$. Although non-linear scaling can be deemed to be involved in the ensemble of iForest, its effect is cancelled by the geometric mean combination. Different anomaly score combination methods have been studied in [13], out of which a method named AOM (Average of Maximum) is shown to be a well-balanced choice and perform very well. Our method can be regarded as a variant of AOM without grouping scores from base detectors. As the non-linear scaling increases the contrast between big values and small values, the arithmetic mean of the scaled values can be dominated by originally big values to a large extent. The extent is controlled by the non-linear function.

Algorithm 4 Subroutine: $path_length(x, \mathcal{F}, node, \eta, L, I_{cur})$ **Input:** Data instance x ; LSH family $\mathcal{F}$; current node $node$; depth adjustment factor η ; granularity level L ; index I .**Output:** The depth of x in the subtree with $node$ as the root.

```

1: if  $node = NULL$  then
2:   return -1;
3: else if  $node.Children = \emptyset$  OR  $I_{cur} > L$  then
4:   return  $I_{cur}(\frac{node.HashIndex}{I_{cur}})^\eta + \mu(node.Size)$ ;
5: else
6:    $K \leftarrow f_{Hash\_Index}(x), f_{Hash\_Index} \in \mathcal{F}$ ;
7:   if  $\exists (K_i : C_i) \in node.Children$  AND  $K = K_i$  then
8:     return  $path\_length(x, \mathcal{F}, C_i, \eta, L, I_{cur} + 1)$ 
9:   else
10:    return  $(I_{cur} + 1)(\frac{node.HashIndex + 1}{I_{cur} + 1})^\eta$ ;

```

Algorithm 4 details $path_length(x, \mathcal{F}, node, \eta, L, I_{cur})$, the recursive subroutine for computing the path length of the instance x from $node$ to the corresponding leaf node. When $I_{cur} = 0$, we can obtain $h(x)$ with respect to a whole tree. The depth level L , $1 \leq L \leq H$, is a user-defined parameter for limiting traversal depth, the same as the height limit in iForest. Isolated anomaly clusters can be detected by lowering the depth level as shown in [8]. Also, we use the term $\mu(node.Size)$ to adjust the path length in Line 4 because a leaf node or an internal node with the depth equal to L may correspond to multiple data instances. The rationale is that the data instances in a larger subtree are more normal than those in a small subtree at the same depth.

Our approach is different from iForest in several aspects. The traversal can stop at both external and internal nodes in our path length computation, as shown in Line 4 and Line 10 of Algorithm 4, while it can only stop at external nodes in iForest. This implies a stronger detection ability of LSHiForest because dissimilar data instances (potential anomalies) will have much shorter path lengths. Another difference is that path lengths of x from both the digital (uncompressed) trie and the PATRICIA (compressed) trie are combined into a single path length, making our method more flexible. Specifically, $h(x) = h_c(\frac{h_u}{h_c})^\eta$, where h_c and h_u are the compressed and uncompressed path length, respectively, and η is an adjustment factor ranging in $[0, 1]$. The parameter η controls the granularity of preserving the distance information in the space defined for the LSH family. When $\eta = 1$, $h(x) = h_u$, implying the finest granularity of isolation is used and the detector can have the best performance for global anomalies. For local anomalies, however, it is desirable to weaken such granularity

because the distances among points in a sparse normal region can be larger than the distance between an anomaly and a dense normal region. Hence, a smaller η can be used for local anomaly detection. Note that this is still insufficient to address the problem of inability to detect local anomalies, but sheds some light on the solutions. A promising solution is to transform the data into another space and apply the isolation based anomaly detection in the transformed space. An instance of our framework in the kernelised space is described in Section IV-D.

With respect to the sample size ψ , the computational complexity of LSHiForest is similar to iForest. Specifically, the average-case time complexity in the training stage is $\Theta(\psi \log_v(\psi))$ and that in the evaluation stage is $\Theta(\log_v(\psi))$. The overall time complexity of the instances with specific LSH families is analysed in Section IV-D. As most LSH functions have $O(1)$ time complexity, our framework can be very fast. Besides, it is very versatile by working with various distance metrics, data spaces and types where an LSH family is defined, and able to detect diverse types of anomaly.

C. Revisiting iForest and SCiForest

In this section, we revisit the two existing tree isolation based anomaly detection methods, i.e., iForest and SCiForest, to study their relationship to our generic framework. Contrary to the claim that these two methods never rely on any distance measure [8], [18], we find that they are indeed associated with certain distance measures. Interestingly, iForest can be directly regarded as an instance of our framework under the ℓ_1 (Manhattan) distance. This also applies to SCiForest under an angular distance without considering its optimisation part.

We first discuss the iForest case. Let $x = \langle x_1, \dots, x_m \rangle$ and $y = \langle y_1, \dots, y_m \rangle$ be two data instances in X . Without loss of generality, we assume that all attributes of X have been normalised into the range $[0, 1]$. Formally, we use the randomised function $f : [0, 1]^m \rightarrow \{-1, 1\}$ to denote the decision-making at each step in iForest, i.e., if $f(x) = -1$, x will be put into the left partition, or else it will be put into the right one. As iForest uniformly randomly chooses an attribute for splitting, $Pr[f(x) = f(y)] = \sum_{i=1}^m Pr[\text{the } i^{th} \text{ attribute is chosen}] \cdot Pr[f_i(x_i) = f_i(y_i)] = \frac{1}{m} \sum_{i=1}^m Pr[f_i(x_i) = f_i(y_i)]$, where $f_i : [0, 1] \rightarrow \{-1, 1\}$ is the randomised function corresponding to the decision after a splitting point $\omega_i \in [0, 1]$ is given for the i^{th} attribute. Specifically, $f_i(x) = \text{sgn}(x - \omega_i)$, where the sign function $\text{sgn}(x) = -1$ if $x < 0$ or 1 if $x \geq 0$. Then, we have $Pr[f_i(x_i) = f_i(y_i)] = 1 - Pr[\omega_i \text{ lies between } x_i \text{ and } y_i] = 1 - |x_i - y_i|$. Finally, we have the following formula:

$$Pr[f(x) = f(y)] = 1 - \frac{1}{m} \sum_{i=1}^m |x_i - y_i|. \quad (3)$$

Note that $\sum_{i=1}^m |x_i - y_i|$ is the ℓ_1 distance $\|x - y\|_1$, a.k.a. the Manhattan distance. Let $d(x, y)$ be the ℓ_1 distance between x and y . The randomised function $f(x)$ is $(d_1, d_2, 1 - \frac{1}{m}d_1, 1 - \frac{1}{m}d_2)$ -sensitive. Therefore, the function family consisting of such hash functions is an LSH family. In essence, iForest is an instance of our framework with such an LSH family.

Let us consider SCiForest, which is essentially an extension of iForest. To avoid axis-parallel partitioning, SCiForest allows using more than one attribute in a splitting step. A randomised function $Q : \mathbb{R}^d \rightarrow \{-1, 1\}$, $1 \leq q \leq m$ is defined over the q attributes for data partitioning. Specifically, $Q(x) = \text{sgn}(\sum_{i=1}^q \omega_i x_i + \omega_0)$, where the coefficient ω_i is chosen randomly from $[-1, 1]$ and ω_0 , called the best splitting point, is determined by an optimisation process. To study the probability $Pr[Q(x) = Q(y)]$, we rewrite $Q(x)$ as $Q(x) = \text{sgn}(\bar{\omega}^T \bar{x})$, where $\bar{\omega} = \langle \omega_0, \omega_1, \dots, \omega_q \rangle$ and $\bar{x} = \langle 1, x_1, \dots, x_q \rangle$. Then, $Pr[Q(x) = Q(y)] = Pr[\text{sgn}(\cos(\theta(\bar{\omega}, \bar{x}))) = \text{sgn}(\cos(\theta(\bar{\omega}, \bar{y})))]$, where $\theta(\bar{\omega}, \bar{x})$ is the angle between the two vectors $\bar{\omega}$ and $\bar{x}$. If we define the distance between x and y as $d(x, y) = \theta(\bar{x}, \bar{y})$, the randomised function $Q(x)$ forms an LSH family based the distance and $Pr[Q(x) = Q(y)] = 1 - \frac{1}{\theta_{max}} \theta(\bar{x}, \bar{y})$. However, it is noted that $\theta_{max} < \pi$ rather than $\theta_{max} = \pi$ in the seminal case [22]. This difference makes choosing $\bar{\omega}$ a non-trivial task, while ω_i can be drawn from the standard Gaussian distribution $\mathcal{N}(0, 1)$ for the seminal angle-based LSH family [22]. Since SCiForest fails to work out how to set the value of ω_0 when letting ω_i , $1 \leq i \leq q$ vary in $[-1, 1]$, it resorts to an optimisation technique. The randomised functions with the optimisation process can be essentially regarded as a learning based hashing scheme, which is data-dependent and has a computational price [20]. Our generic framework can work with learning based hashing schemes without any significant changes.

To show the difficulty in selecting ω_i ($0 \leq i \leq q$) for SCiForest without optimisation, we explore the basic cases when $q = 1$ (similar to iForest) and $q = 2$ (the default setting in [18]). Assume all attributes are normalised to $[0, 1]$, Fig 1(a) illustrates the constraints between ω_0 and ω_1 when $q = 1$. As the normal vector of the hyperplane $\omega_0 x_i + \omega_1 x_1 = 0$, (ω_0, ω_1) must lie within the two hatched triangles in order to make sure the hyperplane varies within $[\frac{\pi}{4}, \frac{\pi}{2}]$, i.e., $\theta_{max} = \frac{\pi}{4}$. Or formally, the inequality $(\omega_0 + \omega_1) \cdot \omega_1 \geq 0$, $\omega_0, \omega_1 \in [-1, 1]$ should hold. The case for $q = 2$ is more complicated. The data space is a square region, the base surface of the inverted pyramid as shown in Fig 1(b), where we centre the region by normalising each attribute into $[-\frac{1}{2}, \frac{1}{2}]$ for better visualisation. Then, $(\omega_0, \omega_1, \omega_2)$ must lie within the other four pyramids whose bases are perpendicular to the X_1 or X_2 axis. Hence, it can be seen that it is a non-trivial task to randomly select ω_i to make $Q(x)$ an LSH hash function. The optimisation technique in [18] mitigates this problem. Note that choosing inappropriate parameters can lead to insufficient distinguishability of hash functions. For example, if the hyperplane in Fig 1(a) can rotate out of $[\frac{\pi}{4}, \frac{\pi}{2}]$, the probability of hashing two data instances into the same bucket cannot be arbitrarily small, which violates the second condition of a locality-sensitive hashing family.

Finally, we have the probability of putting two data instances $x, y \in [0, 1]^m$ into the same partition as $Pr[f(x) = f(y)] = \frac{1}{M} \sum_{i=1}^M Pr[Q_i(\langle x_{i1}, \dots, x_{iq} \rangle) = Q_i(\langle y_{i1}, \dots, y_{iq} \rangle)]$, where $M = \binom{m}{q}$. Based on the above analysis on $Q(x)$, we have

$$Pr[f(x) = f(y)] = 1 - \frac{1}{M} \sum_{i=1}^M \frac{1}{\theta_{max_i}} \theta_i(\bar{x}_i, \bar{y}_i). \quad (4)$$

If we define $d(x, y) = \sum_{i=1}^M \frac{1}{\theta_{max_i}} \theta_i(\bar{x}_i, \bar{y}_i)$ as the distance

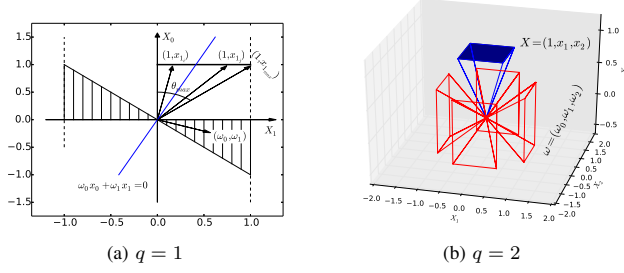


Fig. 1: Examples for constraints on ω_i ($0 \leq i \leq q$) in SCiForest.

between x and y , the randomised function $f(x)$ is $(d_1, d_2, 1 - \frac{1}{M}d_1, 1 - \frac{1}{M}d_2)$ -sensitive and forms an LSH family.

D. Instantiations with Various LSH Families

Besides iForest and SCiForest, in this section we show more instantiations of our generic framework with several commonly-used LSH families. Typically, the LSH families with the ℓ_p ($p = 1, 2$) distance and the angle-based distance in both original and kernelised spaces will be discussed herein.

Unlike iForest and SCiForest, we can have multi-fork LSH trees by using the LSH family under the ℓ_p distance. When $p = 1$, the distance is the ℓ_1 or Manhattan distance which is also associated with iForest, and when $p = 2$, it is the Euclidean distance commonly used in most existing detection methods [5], [6], [13], [16], [17]. The LSH scheme can be achieved by the following randomised hash functions [21]:

$$f_{\omega, \omega_0}(x) = \lfloor \frac{\omega \cdot x + \omega_0}{W} \rfloor, \quad (5)$$

where ω is a random vector with components drawn from the p -stable distribution independently, ω_0 is a real number uniformly sampled from $[0, W]$ and W is a user-specified parameter for the hashing bucket size. Note that the 1-stable distribution is the Cauchy distribution with the density function $c(x) = \frac{1}{\pi} \frac{1}{1+x^2}$, and the 2-stable distribution is the Gaussian (normal) distribution with the density function $g(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$. Although the parameter W can affect the performance of approximate nearest neighbour search, it poses little influence on our approach for anomaly detection because our aim is to isolate data instances, which can be always achieved even if W changes within a broad range of reasonable values.

Although the distance used in SCiForest is also angular, we prefer the seminal one due to its much easier parameter selection. The angle between vector x and y is conventionally defined as $\theta(x, y) = \arccos(\frac{x^T y}{\|x\|_2 \|y\|_2})$. The corresponding LSH function is $f_\omega(x) = \text{sgn}(\omega^T x)$ [22], where ω_i ($1 \leq i \leq m$) is chosen from the standard normal distribution $\mathcal{N}(0, 1)$.

The iForest method has been criticised due to its inability to detect local anomalies [17]. As to our framework, in general its ability to detect local anomalies relies on the specific LSH families although decreasing the adjustment factor η can improve the capability of detecting local anomalies. To enable the

framework to detect local anomalies, we propose to instantiate the framework with kernelised LSH families. Concretely, we apply the tree isolation mechanism in a kernelised space after data are transformed into the space. The rationale is that local anomalies in the original space can be mapped as global anomalies in the kernelised space and become more susceptible to be isolated and detected.

The key is to have a kernelised LSH family. Fortunately, a kernelised angle-based LSH family has been proposed recently [23]. Let $\phi(x)$ and $\kappa(\cdot, \cdot)$ be the map function and the kernel function, respectively. Then, the LSH function is defined as

$$f(\phi(x)) = \text{sgn}\left(\sum_{i=1}^{\lambda} \omega(i) \kappa(x, \hat{x}_i)\right), \quad (6)$$

where $\omega = \bar{K}^{-1/2} e_\xi$, $\bar{K}$ is the $\lambda \times \lambda$ centred kernel matrix formed by $\{\hat{x}_1, \dots, \hat{x}_\lambda\}$ and e_ξ is an $\lambda \times 1$ vector with ones at entries corresponding to the ξ samples. To obtain a family of hash functions, we draw a sample of size λ from the original data set to form the kernel matrix. Note that λ is independent of the sample size ψ for ensemble. Then, ξ entries out of λ is selected for random projection, resulting in $\binom{\lambda}{\xi}$ possible projections. The parameters λ and ξ and the kernel function type will affect the similarity search quality and time efficiency. In practice, λ and ξ can be small enough to achieve fast and high-quality similarity search, e.g., $\lambda = \sqrt{n}$ and $\xi = 30$ [23]. The choice of the kernel function is data-dependent. To produce multi-fork LSH trees, we can use multiple random vectors $[e_{\xi_1} \dots e_{\xi_u}]$ to generate a combined hash key of u bits, leading to the possible maximum branching factor 2^u , $u \geq 1$.

Table III shows the time complexity of the above three instances of our framework. The time complexity of the instances with ℓ_p ($p = 1, 2$) LSH and angle-based LSH is similar to that of iForest, except the dimension factor m . The training time complexity of a kernelised LSH tree consists of three parts, i.e., building the LSH family ($\Theta(\lambda^2 m + \lambda^3)$), transforming training samples into the kernel space ($\psi \lambda m$) and constructing the LSH tree ($\psi \log_v(\psi) \lambda$). Similarly, the time complexity of evaluating a data instance on a kernelised LSH tree consists of two parts, i.e., transforming the data instance into the kernel space (λm) and testing the kernelised data instance on the tree ($\log_v(\psi) \lambda$). Note that all the instances are logarithmic with respect to ψ .

TABLE III: Average-case time complexity of instances.

Method	Training	Predicting
ℓ_1, ℓ_2	$\Theta(t\psi \log_v(\psi)m)$	$\Theta(nt \log_v(\psi)m)$
Angle	$\Theta(t\psi \log_2(\psi)m)$	$\Theta(nt \log_2(\psi)m)$
Kernel	$\Theta(t(\lambda^2 m + \lambda^3 + \psi(\lambda m + \log_v(\psi)\lambda)))$	$\Theta(nt(\lambda m + \log_v(\psi)\lambda))$

One question might be asked: Which is the best LSH family for anomaly detection? The answer is that no LSH family can always beat others and the detection quality is often data-dependent and anomaly-dependent. One key reason is that in real applications anomalies are often defined based on domain-specific knowledge and are associated with certain distance or similarity measures. Therefore, it is necessary to have a diverse range of schemes of the generic framework to cater for different real-world scenarios. Besides the instantiations discussed herein, the framework can be applied anywhere an LSH family is defined. As an increasing number of LSH families have been

developed recently [20], it is quite promising to instantiate our framework accordingly for anomaly detection in the desired distance space for specific applications.

V. EMPIRICAL EVALUATION

A. Experiment Settings

To study the effectiveness and efficiency of our framework against the state-of-the-art methods, in this section we conduct extensive experiments on both synthetic and real-world benchmark data sets. Specifically, we evaluate the aforementioned four instances of our framework, i.e., the angle-based LSH instantiation (ALSH), the ℓ_p ($p = 1, 2$) LSH instantiations (L1SH and L2SH), and the kernelised LSH instantiation (KLSH). To make a comprehensive comparison, we compare our approach with all state-of-the-art subsampling ensemble based anomaly detection methods, i.e., iForest (ISO) [8], SciForest (SCI) [18], average k -NN distance ensemble (EnKNN) [13], LOF ensemble (EnLOF) [13], [16] and iNNE [17].

We set the parameters as follows. For all methods, the number of ensemble members t is set as 100, the same as that used in the existing methods. We use the subsampling technique discussed in Section IV-B for all methods and the sample size varies within the range [64, 1024] which is very close to that used in [13]. For ℓ_p LSH, the bucket size parameter W in (5) is set as 4, the same as the default value used in [21]. We employ the commonly-used RBF (Radial Basis Function) kernel for KLSH. Its performance is sensitive to its bandwidth parameter γ and the optimal selection is often data-dependent. We set γ as 10^8 herein to make use of the ensemble mechanism to reduce the sensitivity, where s is chosen from the uniform distribution $\mathcal{U}(\log_{10}(\frac{0.1}{m}), \log_{10}(\frac{1.0}{m}))$, m is the number of dimensions. The other KLSH parameters λ and ξ in KLSH are set differently for different groups of experiments. We use the same default parameter settings for SciForest as [18], i.e., $q = 2$ and $\tau = 10$. Instead of using a fixed value for the parameter k in EnKNN and EnLOF [13], [16], we let it vary within [1, 10] for each subsample. In this way, we again benefit from the ensemble mechanism to avoid parameter selection for k , whose optimal value is often data-dependent [13].

All algorithms are implemented in Python and the source code can be found at <https://dl.dropboxusercontent.com/u/55237646/LSHiForest-code.zip>. All experiments are executed on a virtual machine with 4 GB main memory, 1 virtual processor and Linux installed as the guest operating system. We gauge the AUC (Area under Curve) and the execution time of each method as the performance indicators. AUC, the area under a ROC curve, is an effective quality measure for anomaly detection and has been extensively used in the literature [8], [13], [16], [17]. A higher AUC value means a better detection quality. The averaged experiment results over 10 runs of each method are reported.

B. Experiment Process and Results

Four groups of experiments are conducted on both synthetic and real-world data sets. We first evaluate how the branching factor v of an LSH forest affects the detection quality. In the second and third groups, we compare the instances of our framework with other methods in the ability to detect surrounded anomalies and local anomalies. Finally, a

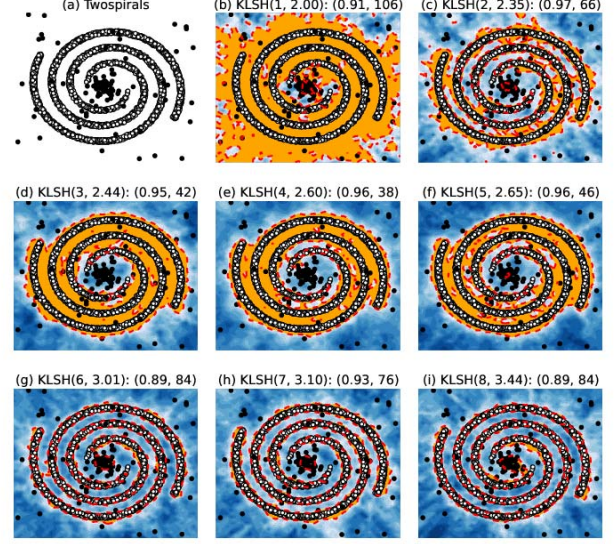


Fig. 2: The effects of average branching factor v on KLSH.

variety of benchmark data sets are used to evaluate detection quality and efficiency of all the aforementioned methods.

1) Effects of Branching Factor: Unlike iForest and SciForest which always produce binary trees, some instances of our framework can produce multi-fork branches, relying on the corresponding LSH families. To study how the branching factor affects the detection quality, we use KLSH with the adjustment factor $\eta = 1$ as a representative for evaluation. The KLSH parameters λ and ξ are set as 300 and 30, respectively, the same as default settings in [23]. In this experiment, we change the key length parameter u within [1, 8] to have u bits of combined hash keys. The real branching factor v usually increases when u becomes larger, and the maximum value can reach 2^u . We set the upper bound of u as 8, which is sufficient for our evaluation because a 1-byte key can produce 256 branches at the maximum. Similar to [17], a synthetic *Twospirals* data set is generated for the evaluation. The 2-dimensional data set has 5000 data instances consisting of two normal spiral patterns, 1% uniformly distributed anomalies and 1% clustered anomalies following the standard Gaussian distribution $\mathcal{N}(0, 1)$. The data set is illustrated in Fig. 2(a). The experimental results are reported in Fig. 2(b)~(i). We plot the contour maps of anomaly scores and the detected boundaries separating the reported anomalies from normal data instances. The real average branching factor, AUC and the number of misclassification errors are also reported in the subtitles with the following format: *method* (u, v): (AUC, errors).

It can be seen from Fig. 2(b)~(i) that the real average branching factor v increases when the hash key length u becomes larger, but is far less than 2^u . Because similar data instances tend to produce the same hash bits with a high probability, the real branching factor seldom increases exponentially with respect to u . Fig. 2(b) shows that the binary case (like in iForest and SciForest) gets a relatively low detection

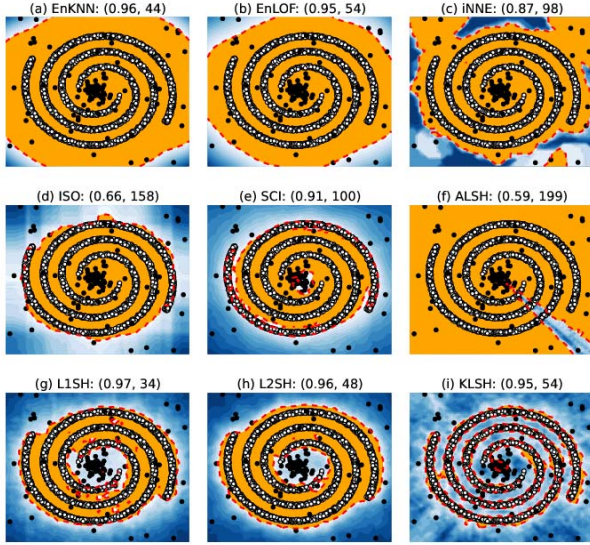


Fig. 3: Capability of detecting surrounded anomalies.

performance and has the most irregular boundary. When v increases, in general the boundaries become increasingly tight to dense regions, implying a stronger capability of isolating data instances in sparse regions from those in dense regions. Note that this has two effects on the detection performance. Local and surrounded anomalies become more susceptible to detect when v increases, while anomaly instances in the relatively dense parts of clustered anomalies have a higher risk of being masked. Therefore, the choice of u (or indirectly v) is often anomaly- or data-dependent. In terms of Fig. 2, choosing u from 2 to 5 can make a good trade-off in practice.

2) *Surrounded Anomalies*: We use the *Twospiral* data set again for this experiment, where most anomalies are well surrounded by the normal spiral patterns. The adjustment factor η is set as 1 for all framework instances. The experimental results are shown in Fig. 3 with subtitle format: method: (AUC, errors). The hash key length u in KLSH is set as 3, and the other KLSH parameters are the same as that just used above.

We can see from Fig. 3 that the behaviour of each method varies differently. Both EnKNN and EnLOF have high detection performance, but broad separation boundary caused by large k (further enlarged by the subsampling effect [13]). The boundary of iNNE is more irregular and affected by global anomalies, because it adopts the hypersphere isolation mechanism and only considers the nearest neighbour ($k = 1$). ISO has poor detection performance and fails to detect the surrounded anomalies. It can be seen from Fig. 3(d) that the contour lines formed by ISO are axis-parallel, consistent with previous empirical studies in [8], [17]. SCI can detect the surrounded anomalies to a certain extent due to its optimisation technique and the use of multiple attributes. ALSH performs the worst. The main reason is that the anomalies are not defined based on the angle distance. Note that this does not mean that ALSH always performs the worst, and it can outperform others

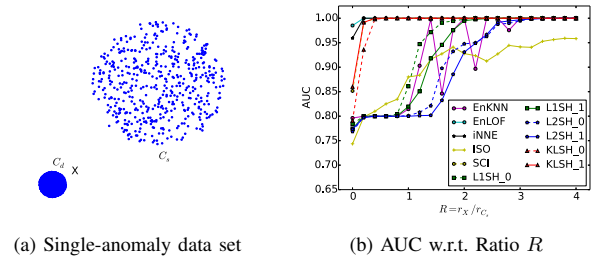


Fig. 4: Capability of detecting local anomalies.

on certain data sets as shown in the fourth experiment. Most importantly, the other three instances, L1SH, L2SH and KLSH, have very high detection performance and tight boundaries separating most surrounded anomalies. This experiment shows that different instances of the generic framework can have different ability to detect surrounded anomalies.

3) *Local Anomalies*: We reuse the single-anomaly synthetic data set in [17], as shown in Fig. 4(a). The data set contains only 1 anomaly and 2499 normal data instances. The normal instances consist of two clusters of different densities. Let C_d and C_s denote the dense and sparse clusters, respectively. To simulate a local anomaly, we put the single anomaly instance around C_d . Let r_{C_s} denote the maximum nearest neighbour distance in C_s , and r_X denote the distance from X to the boundary of C_s . We use the ratio $R = r_X / r_{C_s}$ to signify the locality of an anomaly. Roughly, $R < 1$ implies X is a local anomaly. The ratio varies from 0 to 4 in our experiment. Fig. 4(b) reports the results. The parameters for KLSH are set as follows in terms of [23]: $\lambda = \sqrt{2500} = 50$, $\xi = \frac{\lambda}{4}$, and $\gamma = \frac{1.0}{m}$ (for local anomalies), where $m = 2$. To study how the adjustment factor η affects the performance of our framework, the results with both $\eta = 0$ and $\eta = 1$ are reported. Since the AUC values of ALSH are very low (lower than 0.5), they are excluded from Fig. 4(b) for conciseness.

From Fig. 4(b), it can be observed that only EnLOF, iNNE, SCI and KLSH can detect the anomaly when it is local ($R < 1$). Both EnLOF and iNNE leverage the notion of relative density proposed for local anomaly detection [6], [17]. But note that they have very high computational cost. SCI and KLSH show the capability of our framework in detecting local anomalies. The success of SCI implies instantiations of our framework with learning based hashing have the potential to detect local anomalies. KLSH shows the success of transforming data into kernelised space and applying the isolation mechanism for detecting local anomalies in the original space. Note that both the optimisation process in learning based hashing and the computation of kernel functions in KLSH often incur certain computational cost. Although there is no free lunch for our framework to detect local anomalies, the cost can be much less than EnLOF and iNNE. Another observation is that in the original space the framework instances with $\eta = 0$ have a stronger ability to detect local anomalies than those with $\eta = 1$, adhering to our analysis of Algorithm 4 in Section IV-B. Also note that the AUC of all methods except ISO (and ALSH) reaches 1.00 finally with R increasing, also revealing the shortcomings of ISO. Although both ISO and

TABLE IV: Basic description of data sets.

#	Name	n	m	Outlier vs. Inlier Labels	Rate
1	Glass	214	9	class 6 vs. others	4.2%
2	Ionosphere	351	34	"bad" vs. "good"	35.9%
3	Breastw	683	9	"malignant" vs. "benign"	35.0%
4	Pima	768	8	"positive" vs. "negative"	34.9%
5	Vowel	1456	12	speaker 1 (50 samples) vs. 6, 7 & 8	3.43%
6	Power	4739	4	the 4 th attribute $\geq 1.9 \cdot std$	1.73%
7	Waveform	3505	21	class 0 vs. others	4.62%
8	Wine	5318	11	class 3, 9 & 4 vs. others	4.53%
9	Sensor	5406	24	"Slight-Left-Turn" vs. others	6.07%
10	Occupancy	6841	5	"1" vs. "0"	6.24%
11	Skin	9521	3	"1" vs. "2"	3.67%
12	Satellite	6435	36	class 2, 4 & 5 vs. others	31.6%
13	Shuttle	49097	9	class 2, 3, 5, 6 & 7 vs. class 1	7.15%
14	Coverttype	286048	10	class 4 vs. class 2	0.96%
15	KDDCup99	494021	38	others vs. "neptune", "smurf" & "normal"	1.77%
16	Arrhythmia	452	274	class 3, 4, 5, 7, 8, 9, 14 & 15 vs. others	14.6%
17	Mfeat	666	649	class 4, 8 & 9 (10% samples) vs. others	9.91%
18	Har	5195	561	class 4, 5 & 6 (10% samples) vs. others	10.1%
19	Isolet	6890	617	class 9, 15 & 18 (25% samples) vs. others	4.25%
20	MINST138	20724	784	others (5% samples) vs. "1", "3" & "8"	9.65%

TABLE V: AUC of all methods (%).

#	EnKNN	EnLOF	iNNE	ISO	SCI	ALSH	L1SH	L2SH	KLSH
1	77.26	69.11	83.3	69.34	79.49	89.08	78.22	79.17	82.56
2	93.31	91.04	28.5	83.48	91.11	91.99	84.26	91.23	89.70
3	97.88	37.8	37.8	94.24	98.45	87.75	97.99	97.8	97.53
4	71.48	55.95	55.31	66.34	64.55	60.03	70.17	70.51	70.44
5	95.23	95.09	88.04	79.67	89.68	94.94	80.75	90.64	90.73
6	70.76	70.73	62.35	81.87	48.52	39.63	70.14	77.08	71.74
7	73.6	73.18	61.37	72.06	74.11	51.43	66.72	71.69	73.47
8	68.14	68.43	67.25	64.87	63.3	57.91	65.5	67.05	68.01
9	90.9	84.69	54.43	92.92	92.81	60.22	86.67	89.23	88.25
10	97.36	58.14	44.63	97.28	98.38	89.77	97.79	97.45	97.26
11	78.67	53.15	46.58	61.86	73.94	79.28	72.13	77.03	
12	76.39	60.66	52.97	70.86	70.7	62.18	76.84	77.14	74.47
13	99.38	98.42	62.69	99.32	99.79	74.29	94.89	97.41	89.14
14	92.38	97.95	94.01	89.04	74.11	40.38	87.36	93.64	92.13
15	96.14	30.04	29.93	97.97	98.17	95.75	96.22	96.31	95.43
16	77.35	75.69	70.26	80.5	72.24	65.65	77.16	77.7	80.04
17	94.39	94.53	66.53	90.11	91.38	83.82	70.41	84.15	92.57
18	56.68	63.03	51.91	92.77	99.21	1.52	87.97	94.15	55.74
19	85.97	76.67	51.81	83.62	59.55	45.06	68.45	82.48	87.13
20	84.69	88.22	78.53	80.57	82.07	83.67	75.29	82.48	79.7

L1SH are based on the ℓ_1 distance, L1SH has a stronger ability in local anomaly detection.

4) *Real-world Benchmark Data Sets*: Besides the synthetic data sets, we evaluate the instances of our framework on a variety of real-world benchmark data sets which have been extensively used in the literature [8], [13], [16]–[18]. All the data sets are available at the UCI Machine Learning Repository¹. Most of the data sets are preprocessed and anomalies are labelled the same as the previous work in the literature. All attributes are normalised into Z-scores. The data sets are summarised in Table IV. In this experiment, η is set as 1 for the framework instances. The KLSH parameters λ and ξ are set as follows: $\lambda = \min(\sqrt{n}, 300)$, $\xi = \min(\frac{\lambda}{4}, 30)$.

The AUC of all methods are reported in Table V. For each data set, the leading values are highlighted. The methods with highlighted values can be deemed to have similar performance as their AUC difference is less than 5%. It can be seen from the table that EnKNN and L2SH are the two most robust methods as they perform the best on most of the data sets. SCI, L1SH and KLSH also perform quite well and can be considered stable on most data sets. Although SCI can produce the best AUC values on some data sets, it can also perform nearly the worst on certain data sets, e.g., the *Power* data set. This is caused by its data-dependent optimisation process. In contrast, ISO and all of our framework instances except ALSH have relatively stable performance and can avoid the worst case, attributed to their data-independent hashing feature. ALSH and EnLOF can only perform well on certain data sets and are unstable over all data sets, making their applicability heavily anomaly- or data-dependent. The iNNE method seldom performs well. Note that EnKNN fails to perform well on the high-dimensional data set *Har*, while ISO, SCI, L1SH and L2SH can still work well. This is because the LSH schemes are originally proposed to address the high-dimensional problems in nearest neighbour search. Overall, the instances of our framework can perform robust anomaly detection on a diverse range of data sets, even in high-dimensional space. In particular, L2SH has both advantages of high detection quality and robustness. Although EnKNN has

a comparable performance, it becomes less appealing due to its high computational cost as shown in Table VI.

The execution times of all methods are reported in Table VI. The execution time of EnKNN, EnLOF and iNNE are close to each other, because they have the same core step, i.e., finding the nearest neighbour or top- k nearest neighbours. The time complexity of this operation is linear to the sample size. ISO, ALSH, L1SH and L2SH have similar execution time, attributed to their logarithmic time complexity with respect to the sample size. Roughly, these tree isolation based methods can be faster than the aforementioned three methods by 2 orders of magnitude. Both SCI and KLSH have higher execution time than the data-independent instances of our framework. As the additional computation in SCI is engendered by the training stage, it can still perform efficiently when n becomes very large. In contrast, the execution time of KLSH increases considerably with respect to n as the parameter λ is set as $\min(\sqrt{n}, 300)$. A user can set a smaller value for λ by choosing a better bandwidth parameter γ as the detection performance of KLSH is more sensitive to γ . For high-dimensional data, we can employ the fixed-dimensional feature bagging technique proposed in [13] to enhance the detection efficiency.

In summary, most instances of our framework can achieve high detection performance and time efficiency. In particular, the L2SH instance makes the best trade-off among time efficiency, detection quality and robustness in terms of the experimental results. More importantly, the experiment shows that no instance of our framework can always beat the others due to the variety of anomaly and data types in real-world applications. Therefore, it is a necessity to have the proposed versatile and fast framework for ensemble anomaly analysis.

VI. CONCLUSION

The proliferation of big data applications demands highly scalable and fast data analytics techniques. In this paper, we have investigated the recently developed tree isolation mechanism for fast ensemble anomaly analysis in a principled way. We have analysed the existing tree isolation based detection methods and pointed out their limitation that they fail to generalise to other commonly-used distance measures

¹<https://archive.ics.uci.edu/ml/datasets.html>

TABLE VI: Execution time of all methods (s).

#	EnKNN	EnLOF	iNNE	ISO	SCI	ALSH	LISH	L2SH	KLSH
1	67	71	70	1.99	38	5.41	3.44	5.47	26
2	136	180	148	2.94	53	6.25	4.21	5.9	63
3	357	361	204	3.6	78	8.38	8.02	11	74
4	446	443	405	4.08	82	11	7.37	12	79
5	1019	967	861	5.58	101	14	9.58	16	126
6	2441	2820	2626	12	170	33	36	47	277
7	2232	2063	2059	11	156	31	18	22	335
8	3097	3067	2934	14	208	35	29	38	388
9	3172	3074	3032	12	200	34	43	47	320
10	3917	4349	3896	15	200	42	47	56	470
11	5894	5717	5665	27	260	71	63	91	561
12	4012	3814	3922	17	188	47	29	44	1109
13	29174	25788	23496	127	1058	278	287	372	4082
14	6374	7492	6197	1128	6221	3327	2648	3720	23382
15	12027	13251	9188	3808	11157	5573	5521	6879	76745
16	246	231	223	9.11	87	8.18	4.7	5.53	438
17	454	453	484	22	131	14	7.54	8.68	1415
18	3480	3694	3318	19	228	32	27	20	6534
19	4438	4210	4546	29	268	55	33	37	9415
20	5302	6697	6553	187	658	266	157	170	43502

as the underlying distance measures in these methods have not been well understood. We have proposed a generic framework named LSHiForest for fast tree isolation based ensemble anomaly analysis with the use of a Locality-Sensitive Hashing (LSH) forest. In this way, the fast tree isolation mechanism can apply to any distance or similarity measure, data space or type where an LSH family is defined. The existing methods iForest and SCiForest have been formally shown to be two special cases of our framework. We have further instantiated the framework with three commonly-used LSH families, i.e., ℓ_p ($p = 1, 2$) LSH, angle-based LSH and kernelised LSH. Extensive experiments have been conducted on both synthetic and real-world benchmark data sets to evaluate the instances of the framework and compare with the existing ensemble anomaly detection methods. The experimental results have validated the effectiveness and efficiency of the proposed framework. In particular, the framework instance based on the ℓ_2 LSH family has the best overall performance on the benchmark data sets in terms of time efficiency, detection quality and robustness. In future work, we intend to apply LSHiForest instantiated with certain LSH families in real-time or low-latency anomaly detection applications given its high time efficiency and detection performance.

ACKNOWLEDGMENT

This paper is partially supported by the Open Project of State Key Laboratory for Novel Software Technology KFKT2015A03, the National Science Foundation of China under the grant No. 61672276, and the Australian Research Council Discovery Projects DP170104747 and DP170101932.

REFERENCES

- [1] D. Hawkins, *Identification of Outliers*. Springer Netherlands, 1980.
- [2] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys (CSUR)*, vol. 41, no. 3, p. 15, 2009.
- [3] C. C. Aggarwal, *Outlier Analysis*. Springer New York, 2013.
- [4] L. Duan, L. Xu, Y. Liu, and J. Lee, "Cluster-based outlier detection," *Annals of Operations Research*, vol. 168, no. 1, pp. 151–168, 2009.
- [5] S. Ramaswamy, R. Rastogi, and K. Shim, "Efficient algorithms for mining outliers from large data sets," *ACM SIGMOD Records*, vol. 29, no. 2, pp. 427–438, 2000.
- [6] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," *ACM SIGMOD Record*, vol. 29, no. 2, pp. 93–104, 2000.
- [7] H.-P. Kriegel, A. Zimek *et al.*, "Angle-based outlier detection in high-dimensional data," in *Proc. ACM SIGKDD*, 2008, pp. 444–452.
- [8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM TKDD*, vol. 6, no. 1, p. 3, 2012.
- [9] X. Wu, X. Zhu, G.-Q. Wu, and W. Ding, "Data mining with big data," *IEEE TKDE*, vol. 26, no. 1, pp. 97–107, 2014.
- [10] Z.-H. Zhou, *Ensemble Methods: Foundations and Algorithms*. Chapman & Hall/CRC, 2012.
- [11] C. C. Aggarwal, "Outlier ensembles: Position paper," *ACM SIGKDD Explorations Newsletter*, vol. 14, no. 2, pp. 49–58, 2013.
- [12] A. Zimek, R. J. Campello, and J. Sander, "Ensembles for unsupervised outlier detection: Challenges and research questions a position paper," *ACM SIGKDD Explorations Newsletter*, vol. 15, no. 1, pp. 11–22, 2014.
- [13] C. C. Aggarwal and S. Sathe, "Theoretical foundations and algorithms for outlier ensembles," *ACM SIGKDD Explorations Newsletter*, vol. 17, no. 1, pp. 24–47, 2015.
- [14] S. Geman, E. Bienenstock, and R. Doursat, "Neural networks and the bias/variance dilemma," *Neural Computation*, vol. 4, no. 1, pp. 1–58, 1992.
- [15] A. Lazarevic and V. Kumar, "Feature bagging for outlier detection," in *Proc. ACM SIGKDD*, 2005, pp. 157–166.
- [16] A. Zimek, M. Gaudet, R. J. Campello, and J. Sander, "Subsampling for efficient and effective unsupervised outlier detection ensembles," in *Proc. ACM SIGKDD*, 2013, pp. 428–436.
- [17] T. R. Bandaragoda, K. M. Ting, D. Albrecht, F. T. Liu, and J. R. Wells, "Efficient anomaly detection by isolation using nearest neighbour ensemble," in *Proc. IEEE ICDM Workshop*, 2014, pp. 698–705.
- [18] F. T. Liu, K. M. Ting, and Z. H. Zhou, "On detecting clustered anomalies using SCiForest," in *Proc. ECML-PKDD*, 2010, pp. 274–290.
- [19] M. Bawa, T. Condie, and P. Ganesan, "LSH forest: Self-tuning indexes for similarity search," in *Proc. WWW*, 2005, pp. 651–660.
- [20] J. Wang, H. T. Shen, J. Song, and J. Ji, "Hashing for similarity search: A survey," *arXiv preprint arXiv:1408.2927*, 2014.
- [21] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, "Locality-sensitive hashing scheme based on p-stable distributions," in *Proc. Annual Symp. on Computational Geometry (SCG)*, 2004, pp. 253–262.
- [22] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. STOC*, 2002, pp. 380–388.
- [23] B. Kulis and K. Grauman, "Kernelized locality-sensitive hashing," *IEEE TPAMI*, vol. 34, no. 6, pp. 1092–1104, 2012.
- [24] A. Z. Broder, S. C. Glassman, M. S. Manasse, and G. Zweig, "Syntactic clustering of the web," in *Proc. WWW*, 1997, pp. 1157–1166.
- [25] G. Kollios, D. Gunopulos, N. Koudas, and S. Berchtold, "Efficient biased sampling for approximate clustering and outlier detection in large data sets," *IEEE TKDE*, vol. 15, no. 5, pp. 1170–1187, 2003.
- [26] M. Wu and C. Jermaine, "Outlier detection by sampling with accuracy guarantees," in *Proc. ACM SIGKDD*, 2006, pp. 767–772.
- [27] M. Sugiyama and K. Borgwardt, "Rapid distance-based outlier detection via sampling," in *Proc. NIPS*, 2013, pp. 467–475.
- [28] Y. Wang, S. Parthasarathy, and S. Tatikonda, "Locality sensitive outlier detection: A ranking driven approach," in *Proc. ICDE*, 2011, pp. 410–421.
- [29] M. R. Pillutla, N. Raval, P. Bansal, K. Srinathan, and C. Jawahar, "LSH based outlier detection and its application in distributed setting," in *Proc. CIKM*, 2011, pp. 2289–2292.
- [30] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. STOC*, 1998, pp. 604–613.
- [31] W. Szpankowski, "On the analysis of the average height of a digital trie: Another approach," Department of Computer Science, Purdue University, Tech. Rep. 86-646, 1986.
- [32] W. Szpankowski, "Patricia tries again revisited," *Journal of the ACM (JACM)*, vol. 37, no. 4, pp. 691–711, 1990.